OPERADORES TERNARIO (CONDICIONAL)

Este operador condicional permite elegir entre dos valores dependiendo de si una condición se cumple o no.

Funciona como una pregunta con dos posibles respuestas:

- Si la condición es verdadera (true), se selecciona el primer valor.
- Si la condición es falsa (false), se selecciona el segundo valor.

1. ¿Cómo se escribe?

Su sintaxis es la siguiente:

- condicion ? valorSiVerdadero : valorSiFalso;

Cada símbolo tiene una función:

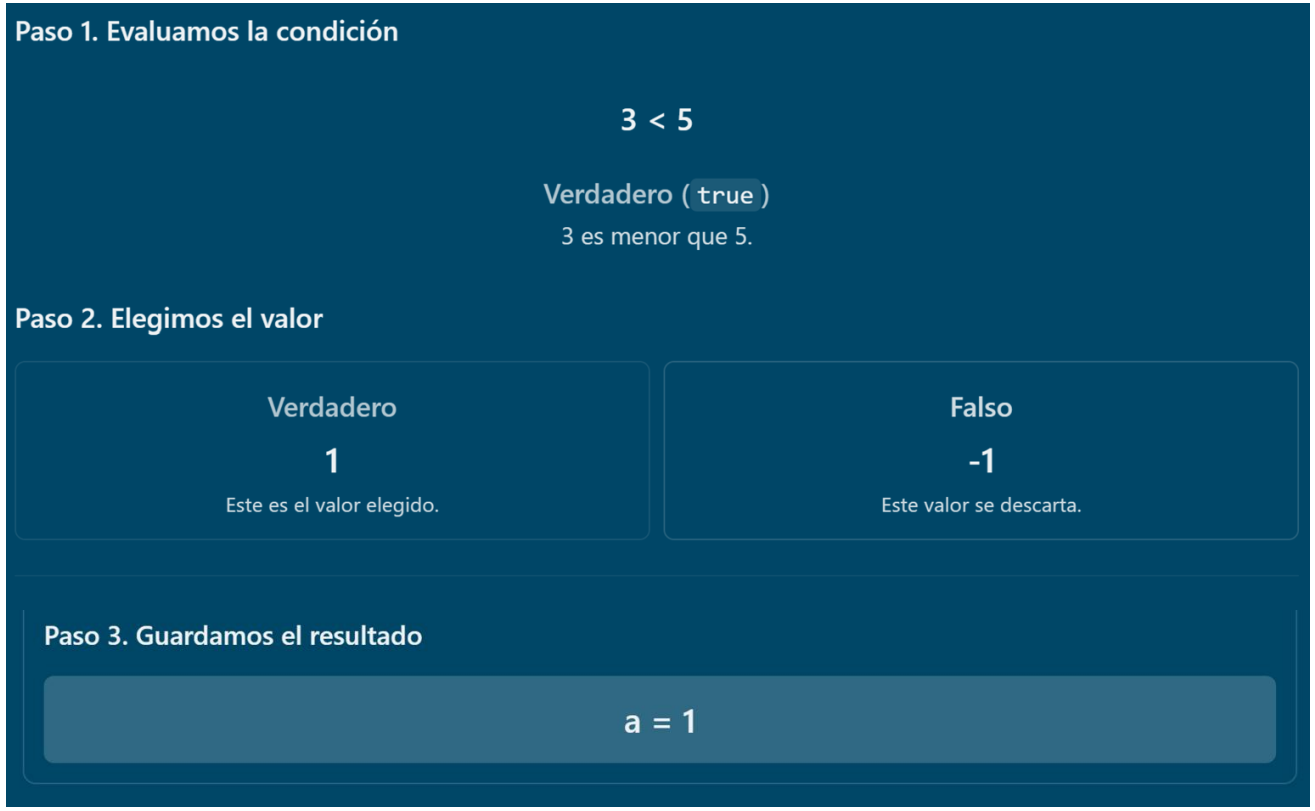
- ? significa: «¿Qué valor elegimos si la condición es verdadera?».
- : significa: «¿Qué valor elegimos si la condición es falsa?».

OPERADORES TERNARIO (CONDICIONAL)

2. Ejemplo sencillo

Imagina que queremos comprobar si un número es mayor que otro y elegir un valor según el resultado.

- **int a;**
- **a = 3 < 5 ? 1 : -1;**



OPERADORES TERNARIO (CONDICIONAL)

Otro ejemplo:

- **int a, b;**
- **a = 3 < 5 ? 1 : -1;**
- **b = a == 7 ? 10 : 20;**

Descripción	Operador
Comparación	== !=

Paso 1. Comprobamos la condición

a == 7

Sabemos que a vale 1.

¿Es 1 igual a 7?

Falso (`false`)

Paso 2. Elegimos el valor

Verdadero	Falso
10	20
No se selecciona.	Este es el valor elegido.

Paso 3. Guardamos el resultado

b = 20

OPERADORES TERNARIO (CONDICIONAL)

Diseña un programa que solicite al usuario una nota numérica entera comprendida entre 0 y 10. A continuación, el programa deberá utilizar el operador ternario para determinar si el alumno ha aprobado o suspendido.

- Si la nota es igual o superior a 5, mostrará el mensaje: Aprobado.
- Si la nota es inferior a 5, mostrará el mensaje: Suspenso.

El programa deberá solicitar la nota por teclado y mostrar el resultado por pantalla. dame el resultado.




```java
package ejercicios;

import java.util.Scanner;

public class Prueba {

    public static void main(String[] args) { // Método principal donde comienza la ejecución del programa.

        // Creamos un objeto Scanner para leer los datos que introduce el usuario.
        Scanner teclado = new Scanner(System.in);

        // Pedimos al usuario que introduzca una nota entre 0 y 10.
        System.out.print("Introduce una nota (0-10): ");

        // Leemos la nota introducida y la guardamos en la variable nota.
        int nota = teclado.nextInt();

        /* Comprobamos si la nota es mayor o igual que 5.
           Si se cumple la condición, resultado será "Aprobado".
           Si no se cumple, resultado será "Suspenso". */
        String resultado = (nota >= 5) ? "Aprobado" : "Suspenso";

        // Mostramos por pantalla el resultado obtenido.
        System.out.println(resultado);

    }
}
```

```
Output - MiPrimerPrograma (run)
run:
Introduce una nota (0-10): 4
Suspenso
BUILD SUCCESSFUL (total time: 3 seconds)
```

```
Output - MiPrimerPrograma (run)
run:
Introduce una nota (0-10): 5
Aprobado
BUILD SUCCESSFUL (total time: 2 seconds)
```

12º ACTIVIDAD PARA HACER EN EL AULA

El enunciado se muestra en el aula.

PRECEDENCIA

En la Tabla 1.6 se muestran los operadores ordenados, según su precedencia, de mayor a menor. En una expresión, la precedencia establece qué operaciones se realizan antes. Con igualdad de precedencia, las operaciones se realizan en el mismo orden en el que se escriben: de izquierda a derecha.

Tabla 1.6. Precedencia de operadores. Los operadores con mayor precedencia (son las primeras operaciones que se realizan) son los posfijos, y los de asignación son los de menor precedencia

Descripción	Operador
Posfijos	`expr++ expr--`
Unarios prefijos	`++expr --expr +expr -expr !expr`
Aritméticos	`* / %`
Aritméticos	`+-`
Relacionales	`< <= > >=`
Comparación	`== !=`
AND lógico	`&&`
OR lógico	`||`
Ternario	`? :`
Asignación	`= += -= *= /= %= &= ^=`

PRECEDENCIA

- **En la expresión:** $2 + 3 * 4$, el operador "$*$" tiene una precedencia mayor que el operador $+$, lo que significa que la multiplicación se realizará antes que la suma. Primero se hace la operación $3 * 4$ (que es 12) y a continuación se realiza la suma $2 + 12$ (que es 14).
- **En cambio, en la expresión:** $3 <= 5 \&\& 2 == 2$, el operador con mayor precedencia es $<=$, que será la primera operación que se realice, siendo cierta.
  - Queda: true && 2 == 2
  - A continuación, se realizará la comparación, que también resulta cierta: true && true
  - Y el operador con menor precedencia de los tres es el *AND lógico*, que se realiza en último lugar, por lo que la expresión resulta cierta.

IMPORTANTE: La precedencia puede romperse utilizando paréntesis; por ejemplo, en la expresión:

(2 + 3) * 4, el uso de paréntesis obliga a que la primera operación que se realice sea la suma.

Descripción	Operador
Posfijos	`expr++ expr--`
Unarios prefijos	`++expr --expr +expr -expr !expr`
Aritméticos	`* / %`
Aritméticos	`+-`
Relacionales	`< <= > >=`
Comparación	`== !=`
AND lógico	`&&`
OR lógico	`||`
Ternario	`? :`
Asignación	`= += -= *= /= %= &= ^=`

PRECEDENCIA

Posfijo: x++

El ++ está después de la variable.

- Primero usa el valor que tiene.
- Después suma 1.

Ejemplo:

x = 5;

y = x++;

Resultado:

y = 5

x = 6

"Dame el valor de “x” y luego ya le sumas 1".

Descripción	Operador
Posfijos	expr++ expr--
Unarios prefijos	++expr --expr +expr -expr !expr
Aritméticos	* / %
Aritméticos	+-
Relacionales	< <= > >=
Comparación	== !=
AND lógico	&&
OR lógico	||
Ternario	? :
Asignación	= += -= *= /= %= &= ^=

PRECEDENCIA

Prefijo: ++x

El ++ está **antes** de la variable.

- Primero suma 1.
- Después usa el nuevo valor.

Ejemplo:

x = 5;

y = ++x;

Resultado:

y = 6

x = 6

"Primero súmale 1 a “x” y luego me das el resultado".

Descripción	Operador
Posfijos	expr++ expr--
Unarios prefijos	++expr --expr +expr -expr !expr
Aritméticos	* / %
Aritméticos	+-
Relacionales	< <= > >=
Comparación	== !=
AND lógico	&&
OR lógico	||
Ternario	? :
Asignación	= += -= *= /= %= &= ^=

CONVERSIÓN DE TIPOS

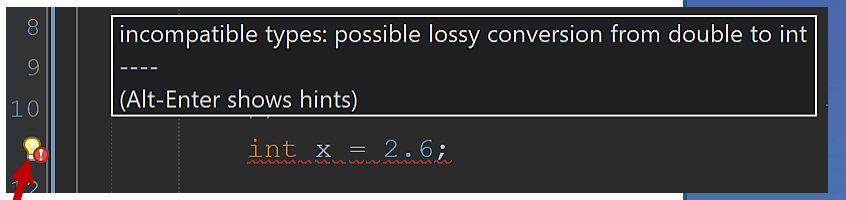
Como hemos visto, todas las variables en Java tienen asociado un tipo. Cuando asignamos un valor a una variable, ambos deben ser del mismo tipo. A una variable de tipo int se le puede asignar un valor int y a una variable double se le puede asignar un valor double.

- **int a = 2;**
- **double x = 2.3;**

Cada tipo se caracteriza por ocupar un tamaño en memoria, donde se almacenan los valores correspondientes.

Si escribimos

- **int a = 2.6;** //trata de asignar un valor real a una variable entera.



El compilador nos avisará de que estamos cometiendo un error y no nos dejará ejecutar. La razón de este control de tipos tan estricto es evitar errores durante la ejecución del programa, ya que es evidente que una variable de un tipo no puede almacenar valores con un tamaño superior.

Por ejemplo, un valor double ocupa en la memoria 64 bits, mientras que una variable int utiliza 32 bits para almacenar un valor. Simplemente un valor double no cabe en una variable int.

CONVERSIÓN DE TIPOS

Sin embargo, un valor int puede guardarse sin problemas en una variable double. ¿Por qué no permitir una asignación como esta?

- **int a = 3;**
- **double x = a;**

Java permite esta asignación sin violar la norma de que a una variable double se le asigne un valor double.

Para ello, Java convierte de forma automática el valor entero 3 en el valor double 3.0 antes de asignarlo a la variable x. Esto es posible porque la variable double es de mayor tamaño que el valor int, es decir, tiene suficiente espacio para guardarlo.

Por tanto, este tipo de conversiones y asignaciones automáticas será posible cuando la variable sea de mayor tamaño que el del valor asignado. Se habla entonces de conversiones de ensanchamiento. Nos permitirá, por ejemplo, guardar valores byte, short, int, long o float en una variable double.

Muy distinto es que intentemos asignar un valor double en una variable int, ya que no tiene sitio suficiente para guardarlo. Lo normal es que se trate de un error del programador.

En este caso Java no hará ninguna conversión automática. Se limitará a darnos un error de compilación.

CONVERSIÓN DE TIPOS

Sin embargo, a veces es interesante guardar la parte entera de un número con decimales en una variable entera. Evidentemente, esto supone una pérdida de información, ya que los decimales desaparecerán.

Para ello deberemos colocar un *cast* o molde delante del valor que queremos asignar,

- **int a = (int) 2.6;** //(int) indica el tipo al que se convertirá el valor

El *cast* es lo que va entre paréntesis. Lo que hace es eliminar (truncar) la parte decimal de 2.6 y convertirlo en el entero 2, que podrá ser asignado a la variable a sin problemas.

Este tipo de conversión se llama *de estrechamiento*, ya que fuerza la asignación de un tipo de dato en una variable de menor tamaño, eso sí, con pérdida de información.

Nada impide, aunque no es necesario, colocar un *cast* en una conversión de ensanchamiento. Esto a veces hace que el programa gane en legibilidad:

- **double x = (double) 3;**

CONVERSIÓN DE TIPOS

Nota técnica

Caso especial de conversión son los valores de tipo `char` que, con un tamaño de 16 bits, pueden convertirse a su valor entero en el código UNICODE, como se verá más adelante. Esto permite asignarlos a una variable `int`:

```
int c = 'a'; //que equivale a int c = 97;
```

ya que 97 es el código asignado al carácter 'a'.

Lou Ascii

```
000:   013:♪  026:→  039:'  052:4  065:A  078:N  091:[  104:h  117:u
001:☺  014:♫  027:←  040:(  053:5  066:B  079:O  092:\  105:i  118:v
002:☻  015:☼  028:∟  041:)  054:6  067:C  080:P  093:]  106:j  119:w
003:♥  016:►  029:↔  042:*  055:7  068:D  081:Q  094:^  107:k  120:x
004:♦  017:◄  030:▲  043:+  056:8  069:E  082:R  095:_  108:l  121:y
005:♣  018:↕  031:▼  044:,  057:9  070:F  083:S  096:`  109:m  122:z
006:♠  019:‼  032:   045:-  058::  071:G  084:T  097:a  110:n  123:{
007:•  020:¶  033:!  046:.  059:;  072:H  085:U  098:b  111:o  124:|
008:◘  021:§  034:"  047:/  060:<  073:I  086:V  099:c  112:p  125:}
009:○  022:▬  035:#  048:0  061:=  074:J  087:W  100:d  113:q  126:~
010:◙  023:↨  036:$  049:1  062:>  075:K  088:X  101:e  114:r  127:⌂
011:♂  024:↑  037:%  050:2  063:?  076:L  089:Y  102:f  115:s
012:♀  025:↓  038:&  051:3  064:@  077:M  090:Z  103:g  116:t
```

```
1  package ejercicios;
2
3  public class Prueba {
4
5      public static void main(String[] args) {
6
7          int c = 'c';
8
9          System.out.println("Valor de c: " + c);
10
11     }
12 }
```

```
run:
Valor de c: 99
BUILD SUCCESSFUL (total time: 0 seconds)
```

13º ACTIVIDAD PARA HACER EN EL AULA

El enunciado se muestra en el aula.

CONVERSIÓN DE TIPOS: REDONDEO

En Java, una forma de redondear números positivos al entero más cercano consiste en **sumar 0.5 al número y convertir el resultado al tipo int**.

La conversión a int elimina la parte decimal del resultado.

La instrucción es:

- redondeo = (int) (numero + 0.5);

Por ejemplo, si numero contiene el valor 4.8, se suma 0.5, obteniendo 5.3. Después, al convertirlo a int, se eliminan los decimales y se obtiene 5.

- double numero = 4.8;
- int redondeo;
- redondeo = (int) (numero + 0.5);
- System.out.println(redondeo); // Imprime 5

Importante: este procedimiento se utiliza para números positivos. Para redondear números negativos correctamente según las reglas de Java, es preferible utilizar **Math.round().**

Redondeo mediante Math.round()

Java dispone del método Math.round(), que permite redondear un número al entero más cercano sin necesidad de sumar manualmente 0.5.

- Si recibe un **double**, Math.round() devuelve un **long**.
- Si recibe un **float**, Math.round() devuelve un **int**.

Y si recibo un double y quiero que me devuelva un int en vez de un long ¿cómo lo haría?

- double numero = 4.8;
- int redondeo;
- redondeo = (int) Math.round(numero);
- System.out.println(redondeo); // Imprime 5

Recuerda: el (int) no forma parte de Math.round(). Es una conversión de tipo (casting) que añadimos porque queremos guardar el resultado en una variable int.

Redondeo mediante Math.round()

En Java, también podemos redondear un número decimal para conservar únicamente una, dos, tres o más cifras decimales.

Para ello, podemos utilizar el método Math.round() junto con una multiplicación y una división.

El procedimiento consiste en multiplicar el número por una potencia de 10, redondear el resultado y dividirlo por el mismo valor.

La fórmula general es:

```
resultado = Math.round(numero * factor) / (double) factor;
```

El factor dependerá del número de cifras decimales que queramos conservar:

Cifras decimales	Factor	Ejemplo
1	10.0	3.456 → 3.5
2	100.0	3.456 → 3.46
3	1000.0	3.4567 → 3.457
4	10000.0	3.45678 → 3.4568

Redondeo mediante Math.round()

Ejemplo: redondear a tres cifras decimales

- double numero = 3.4567;
- double redondeo;
- redondeo = Math.round(numero * 1000.0) / 1000.0;
- System.out.println(redondeo); // Imprime 3.457

¿Cómo funciona?

1. Multiplicamos el número por 1000: obtenemos 3456.7.
2. Aplicamos Math.round(): obtenemos 3457.
3. Dividimos entre 1000.0: obtenemos 3.457.

¿Por qué funciona sin (double)?

1. Math.round(numero * 1000.0) devuelve un long con el valor 3457.
2. Al dividir entre 1000.0, que es un double, Java realiza una división decimal.
3. El resultado 3.457 se guarda en la variable redondeo, de tipo double.

No hace falta convertir el resultado a double manualmente porque Java lo hace automáticamente al dividir entre un número de tipo double.

Regla para recordar

Para conservar un número determinado de cifras decimales, multiplicamos y dividimos por una potencia de 10:

- **Una cifra decimal: 10.0**
- **Dos cifras decimales: 100.0**
- **Tres cifras decimales: 1000.0**
- **Cuatro cifras decimales: 10000.0**

Importante: utilizamos 1000.0 en lugar de 1000 para que la división sea decimal y no una división entera. Este procedimiento redondea el valor numérico, pero no garantiza que se muestren siempre todos los ceros finales. Si necesitamos mostrar exactamente dos cifras decimales, podemos utilizar System.out.printf("%.2f", numero);.

Redondeo mediante Math.round()

No debemos confundir los siguientes conceptos:

- **Truncar:** elimina la parte decimal sin tenerla en cuenta. Por ejemplo, (int) 4.8 devuelve 4.
- **Redondear:** aproxima el número al entero más cercano. Por ejemplo, Math.round(4.8) devuelve 5.

En resumen, truncar elimina los decimales, mientras que redondear tiene en cuenta su valor para obtener una aproximación.



14º ACTIVIDAD PARA HACER EN EL AULA

El enunciado se muestra en el aula.

ACTIVIDADES FINALES DE APTO/NO APTO

El enunciado se muestra en el aula.

ACTIVIDADES FINALES DE APTO/NO APTO

El enunciado se muestra en el aula.